# MINI PROJECT 2: Data Mining and Machine Learning


## Mohammed Imtiyaz

## FAU ID:Z23749265


---

1. **a. Download the data file 'digits.mat' from Canvas.**
   **b.Load this data file to your program (in Matlab type: load digits.mat)**

   **c.You will have four variables:**

| Name | Size | Bytes | Class | Attributes |
|---|---|---|---|---|
| test | 784x1000 | 784000 | uint8 | |
| testlabels | 1000x1 | 1000 | uint8 | |
| train | 784x5000 | 3920000 | uint8 | |
| trainlabels | 5000x1 | 5000 | uint8 | |

   **The variable 'train' includes images of 5000 hand-written digits. However, to see images of these digits you should change 'train' from a 784 x 5000 to a variable such as 'x' which has the dimension of 28 x 28 x 5000. Then you will have five thousand 28 x 28 images of these digits. A simple way of doing this is:**

```
load digits.mat for k = 1:5000      dummy = train(:,k) ;      for
i = 1:28          for j = 1:28              x(i,j,k) =
double(dummy((i-1)*28 + j)) ;                    end      end
imagesc(x(:,:,k)')      colormap('gray')      colorbar
pbaspect([1 1 1])      pause(0.2) end
```

   **The number associated with each digit in 'train' is given in 'trainlabels'.**
   **In this project, you should design and train a single-layer 2-dimensional Perceptron to recognize these digits.**

   d. **Design and train a single-layer perceptron that can detect the digit '0'. In other words, your perceptron should return +1 when the digit in the image is '0' and -1 otherwise. Train your perceptron with 5000 images available in 'train' and test the performance of your system by using the other variable, 'test', in your data file. When you are training your perceptron, simultaneously, monitor the number of errors in the last 100 iterations. (Try to choose a reasonable value for your training rate and reduce it as a function of iteration number). How is the performance of the system? Also look at the image of your weights 'w' which is a 28 x 28 variable vector. How does it look? Why?**

   e. **Repeat this experiment, but this time try to detect digit '8'. Does your network perform better or worse? Why? Now try '1' and '2'.**

Sol:

In a single-layer perceptron, the number of neurons refers to the number of units in the layer. Each neuron corresponds to a single unit in the layer In a single-layer perceptron, there is typically no hidden layer. The term "single-layer" refers to the fact that there is only one layer of neurons, which directly connects the input to the output..

we are designing a single-layer perceptron to detect the digit '0'. Here's how the number of neurons is determined:

Input layer: The input layer represents the pixels of the input images. Each pixel is considered as an input feature. Since the images are 28x28 pixels, there are a total of 28*28 = 784 input neurons.

Output layer: In this task, we want the perceptron to detect whether the input image represents the digit '0' or not. Therefore, we only need a single output neuron. This neuron will output a value indicating whether the input image is a '0' (output value +1) or not (output value -1).

So, in this single-layer perceptron:

The input layer has 784 neurons (corresponding to the 28x28 = 784 input features).

The output layer has 1 neuron (corresponding to the binary classification task).

Python code for the same:

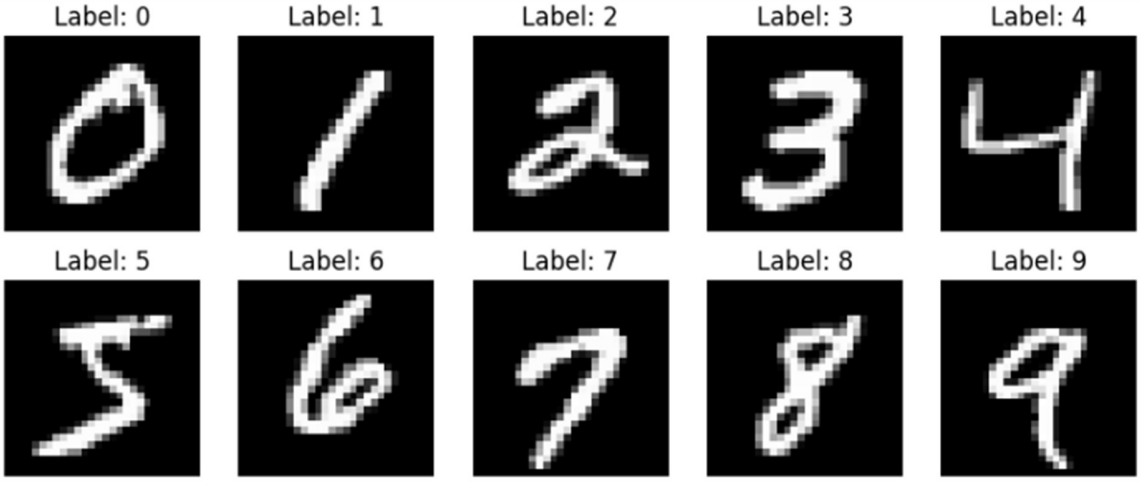
For displaying images:

```
import numpy as np
import scipy.io
import matplotlib.pyplot as plt

# Load data
data = scipy.io.loadmat('/content/drive/MyDrive/digits.mat')

# Extract variables
train = data['train'].T
trainlabels = data['trainlabels'].flatten()
test = data['test'].T
testlabels = data['testlabels'].flatten()

# Display 10 distinct images along with their labels
unique_labels = np.unique(trainlabels)
plt.figure(figsize=(10, 4))
for i, label in enumerate(unique_labels[:10]):
    idx = np.where(trainlabels == label)[0][0]
    image = train[idx].reshape(28, 28)
    plt.subplot(2, 5, i+1)
    plt.imshow(image.T, cmap='gray')
    plt.title("Label: " + str(label))
    plt.axis('off')
plt.show()
```



For 0:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat
from google.colab import drive
drive.mount('/content/drive')

# Load the data file
data = loadmat('/content/drive/MyDrive/digits.mat')

train = data['train']
trainlabels = data['trainlabels'].flatten()
test = data['test']
testlabels = data['testlabels'].flatten()

# Convert labels: +1 for the digit to detect, -1 for all other digits
digit_to_detect = 0
train_labels = np.where(trainlabels == digit_to_detect, 1, -1)
test_labels = np.where(testlabels == digit_to_detect, 1, -1)

# Flatten images for perceptron training
train_flattened = train.T
test_flattened = test.T

def train_perceptron(X, y, learning_rate=0.01, epochs=10):
    weights = np.zeros(X.shape[1] + 1)
    errors = []

    for epoch in range(epochs):
        error_count = 0
        for xi, target in zip(X, y):
            xi = np.insert(xi, 0, 1)  # Bias integration
            update = learning_rate * (target - predict(xi, weights))
            weights += update * xi
            error_count += int(update != 0)
        errors.append(error_count)

        # Dynamically adjust learning rate if desired
        learning_rate /= 1.02  # Example of reducing the learning rate

    return weights, errors

def predict(xi, weights):
    return np.where(np.dot(xi, weights) >= 0, 1, -1)

def evaluate_perceptron(X, y, weights):
    X_bias = np.hstack([np.ones((X.shape[0], 1)), X])  # Add bias term
    predictions = predict(X_bias, weights)
    accuracy = np.mean(predictions == y)
```

```
    error_rate = 1 - accuracy  # Calculate error rate
    return accuracy, error_rate

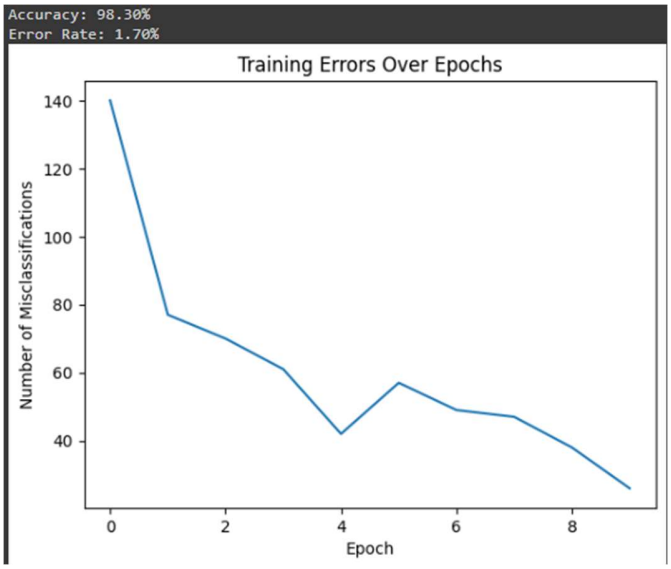
def visualize_weights(weights):
    plt.imshow(weights[1:].reshape(28, 28), cmap='gray')  # Skip bias weight
    plt.colorbar()
    plt.title('Perceptron Weights Visualization')
    plt.show()

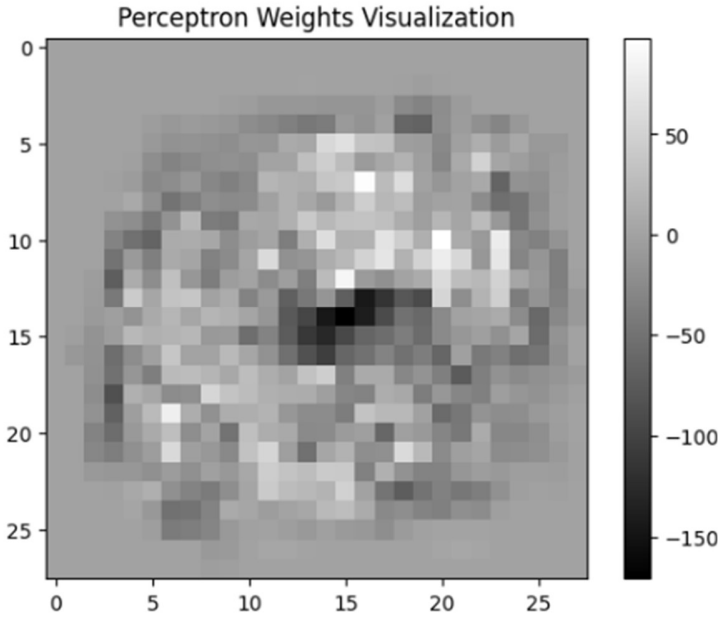
# Train the perceptron
weights, errors = train_perceptron(train_flattened, train_labels, learning_rate=0.01,
epochs=10)

# Evaluate the perceptron
accuracy, error_rate = evaluate_perceptron(test_flattened, test_labels, weights)
print(f'Accuracy: {accuracy*100:.2f}%')
print(f'Error Rate: {error_rate*100:.2f}%')

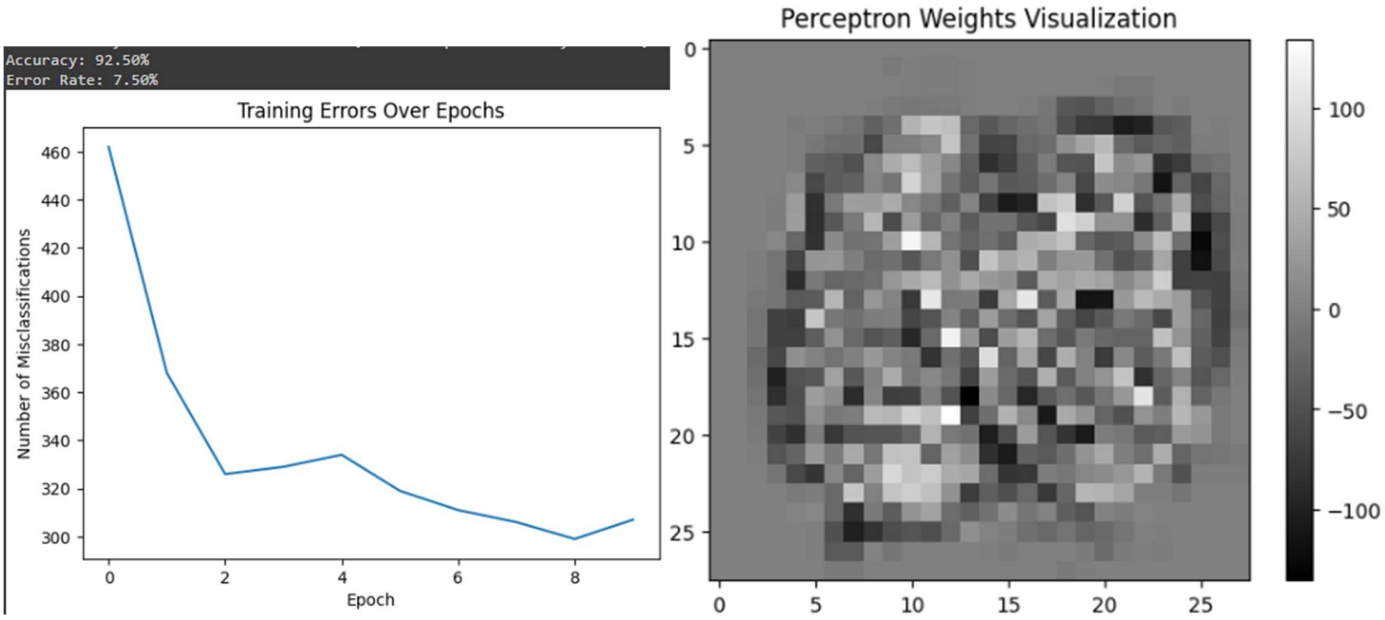
# Visualize training errors and weights
plt.plot(errors)
plt.title('Training Errors Over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Number of Misclassifications')
plt.show()

visualize_weights(weights)
```



for digit 8: changing the digit_to_detect = 8, we get the following output

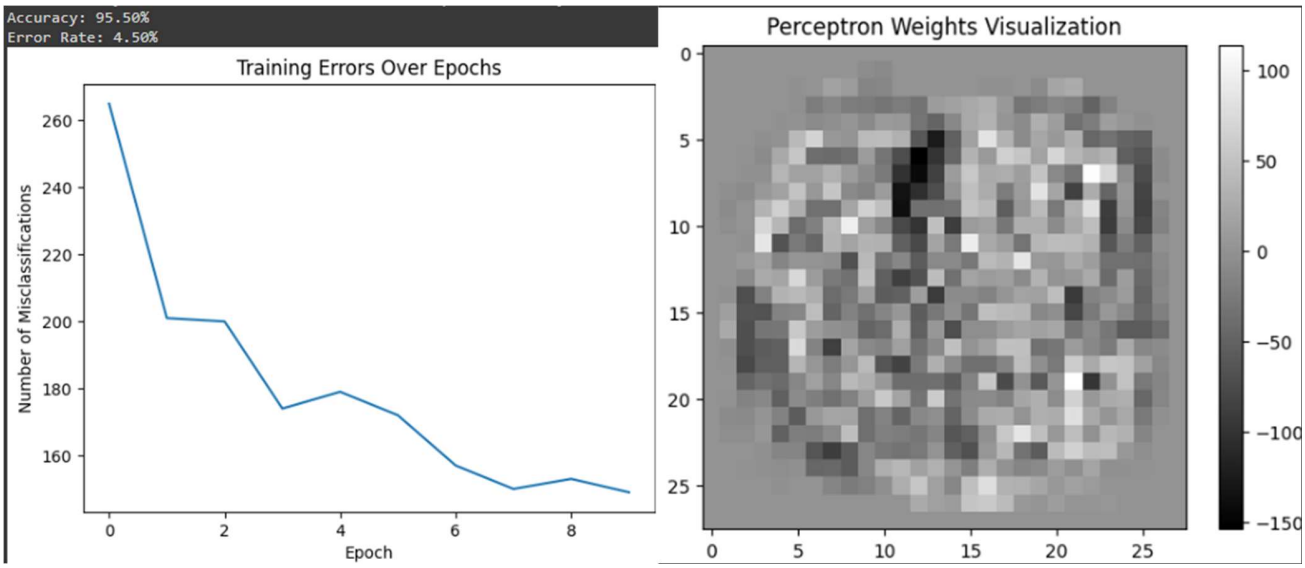



The performance of the system for detecting the digit '8' is reported as an error rate of 7.50%. This means that approximately 7.50% of the test samples labeled as '8' were misclassified by the perceptron.

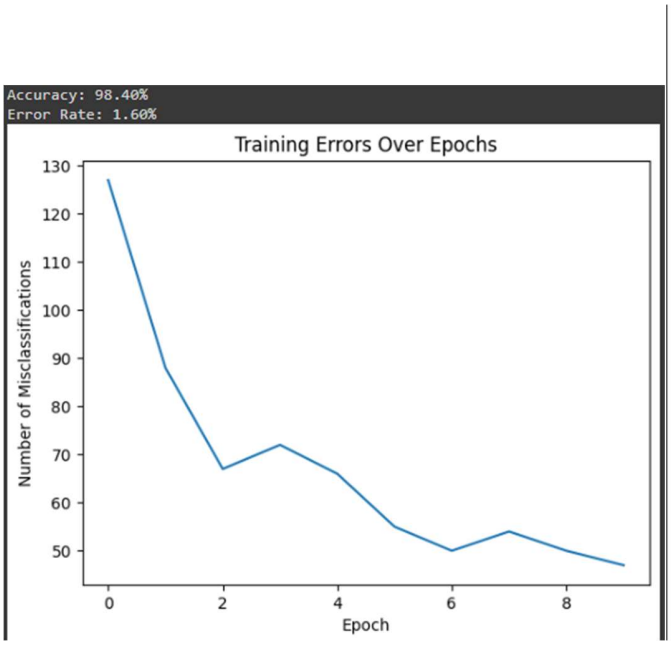
Comparing this error rate with the performance of the system for detecting digit '0' (which had an error rate of 1.70% as reported earlier), we can observe that the system performs relatively worse for detecting digit '8'. This suggests that the perceptron may have more difficulty distinguishing digit '8' from other digits compared to digit '0'.

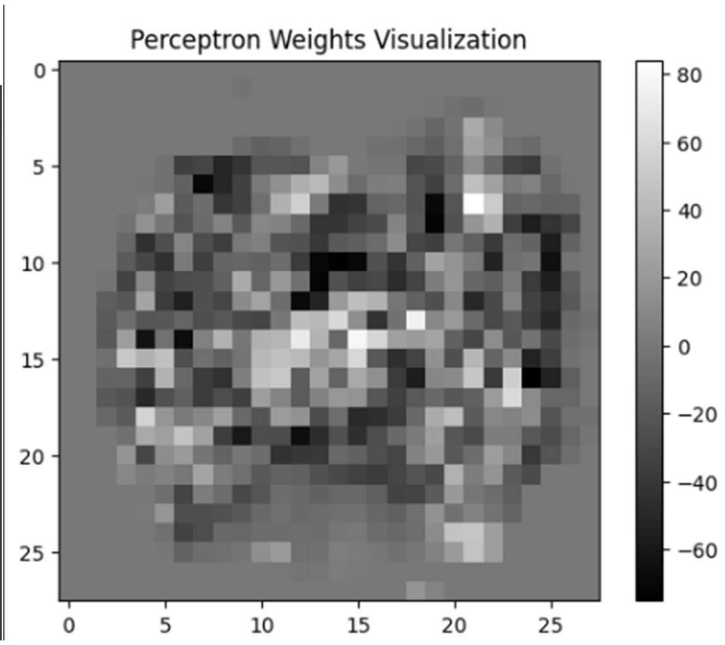
Further analysis and optimization may be needed to improve the performance of the system for detecting digit '8', such as adjusting the learning rate, increasing the number of iterations, or using more complex models.

For digit 2: changing the digit_to_detect = 2, we get the following output



`For digit1:` changing the digit_to_detect = 1, we get the following output




Here's a summary of the error rates for each digit:

Error rate for detecting digit '0': 1.70%

Error rate for detecting digit '1': 1.60%

Error rate for detecting digit '2': 4.50%

Error rate for detecting digit '8': 7.50%

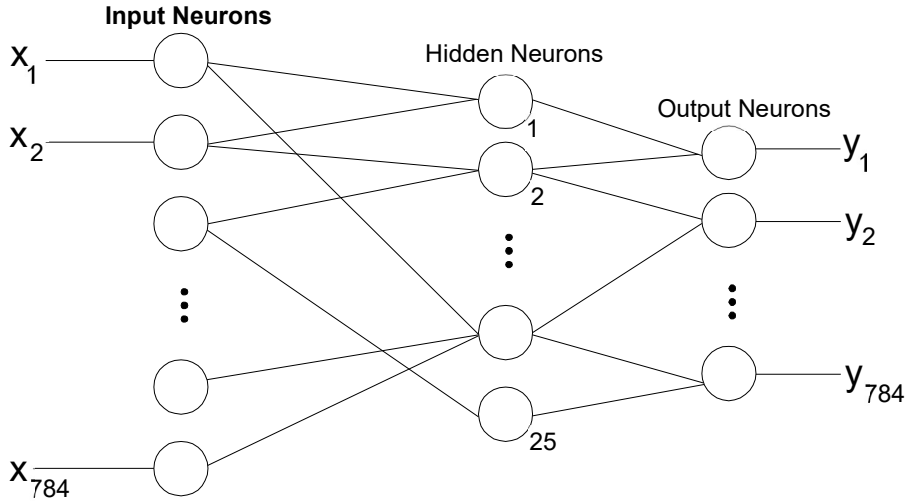
Based on these error rates, we can observe the following:

The perceptron performs relatively well in detecting digits '0' and '1', with error rates of 1.70% and 1.60% respectively. These error rates are relatively low, indicating good performance in correctly classifying these digits.

For digit '2', the error rate increases to 4.50%, indicating a slightly higher misclassification rate compared to digits '0' and '1'.

Digit '8' has the highest error rate among the reported digits, with an error rate of 7.50%. This suggests that the perceptron has more difficulty distinguishing digit '8' from other digits, resulting in a higher misclassification rate.

**2. Consider the following two-layer Perceptron. We have 784 input neurons, 25 hidden neurons, and 10 output neurons in this network. We want to use this network for digit recognition like what you did in problem 1. However, this time we have 10 output neurons. Neuron number i (i=0,1,2,...,9) should return 1 when the digit in the input pattern is 'i' and 0 otherwise. For example, if the input pattern represents number '2' the output vector should look like [0,0,1,0,0,0,0,0,0,0]. Therefore, in your project you prepare a computer code to train the network by adjusting weights for this task. The algorithm that we use in this project is BACK-PROPAGATION algorithm.**




**Sol:**

```
import numpy as np
from scipy.io import loadmat
import matplotlib.pyplot as plt

# Loading the dataset
data = loadmat('/content/drive/MyDrive/digits.mat')  # Update the path as needed
x_train = data["train"].T / 255.0  # Normalizing the data
x_test = data["test"].T / 255.0
y_train = data["trainlabels"].flatten()
y_test = data["testlabels"].flatten()

# Activation functions and their derivatives
def ReLU(x):
    return np.maximum(0, x)

def der_ReLU(x):
    return np.where(x > 0, 1, 0)

def Softmax(x):
    exp_values = np.exp(x - np.max(x, axis=1, keepdims=True))
    probabilities = exp_values / np.sum(exp_values, axis=1, keepdims=True)
    return probabilities

# Convert labels to one-hot encoding
def to_categorical(y, num_classes=10):
    return np.eye(num_classes)[y]
```

```
# Class for the MultiLayer Perceptron
class MultiLayerPerceptron:
    def __init__(self, lr, iterations, n_hidden_neurons, n_output_neurons, n_inputs):
        np.random.seed(0)
        self.lr = lr
        self.iterations = iterations
        self.weights_hidden = np.random.randn(n_inputs, n_hidden_neurons) *
np.sqrt(2. / n_inputs)
        self.weights_output = np.random.randn(n_hidden_neurons, n_output_neurons) *
np.sqrt(1. / n_hidden_neurons)
        self.bias_hidden = np.zeros((1, n_hidden_neurons))
        self.bias_output = np.zeros((1, n_output_neurons))
        self.loss_history = []

    def forward(self, X):
        self.Z1 = np.dot(X, self.weights_hidden) + self.bias_hidden
        self.A1 = ReLU(self.Z1)
        self.Z2 = np.dot(self.A1, self.weights_output) + self.bias_output
        self.A2 = Softmax(self.Z2)
        return self.A2

    def backprop(self, X, y):
        m = y.shape[0]
        dZ2 = self.A2 - y
        dW2 = np.dot(self.A1.T, dZ2) / m
        db2 = np.sum(dZ2, axis=0, keepdims=True) / m

        dZ1 = np.dot(dZ2, self.weights_output.T) * der_ReLU(self.Z1)
        dW1 = np.dot(X.T, dZ1) / m
        db1 = np.sum(dZ1, axis=0, keepdims=True) / m

        # Update the weights and biases
        self.weights_hidden -= self.lr * dW1
        self.weights_output -= self.lr * dW2
        self.bias_hidden -= self.lr * db1
        self.bias_output -= self.lr * db2

    def train(self, X, y):
        y_one_hot = to_categorical(y)
        for epoch in range(self.iterations):
            output = self.forward(X)
            self.backprop(X, y_one_hot)
            loss = -np.mean(np.log(output[np.arange(len(y)), y] + 1e-9))
            self.loss_history.append(loss)
            if epoch % 10 == 0:
                print(f'Epoch {epoch}, Loss: {loss}')

    def predict(self, X):
        predictions = self.forward(X)
```

```
        return np.argmax(predictions, axis=1)

    def accuracy(self, X, y):
        predictions = self.predict(X)
        return np.mean(predictions == y) * 100

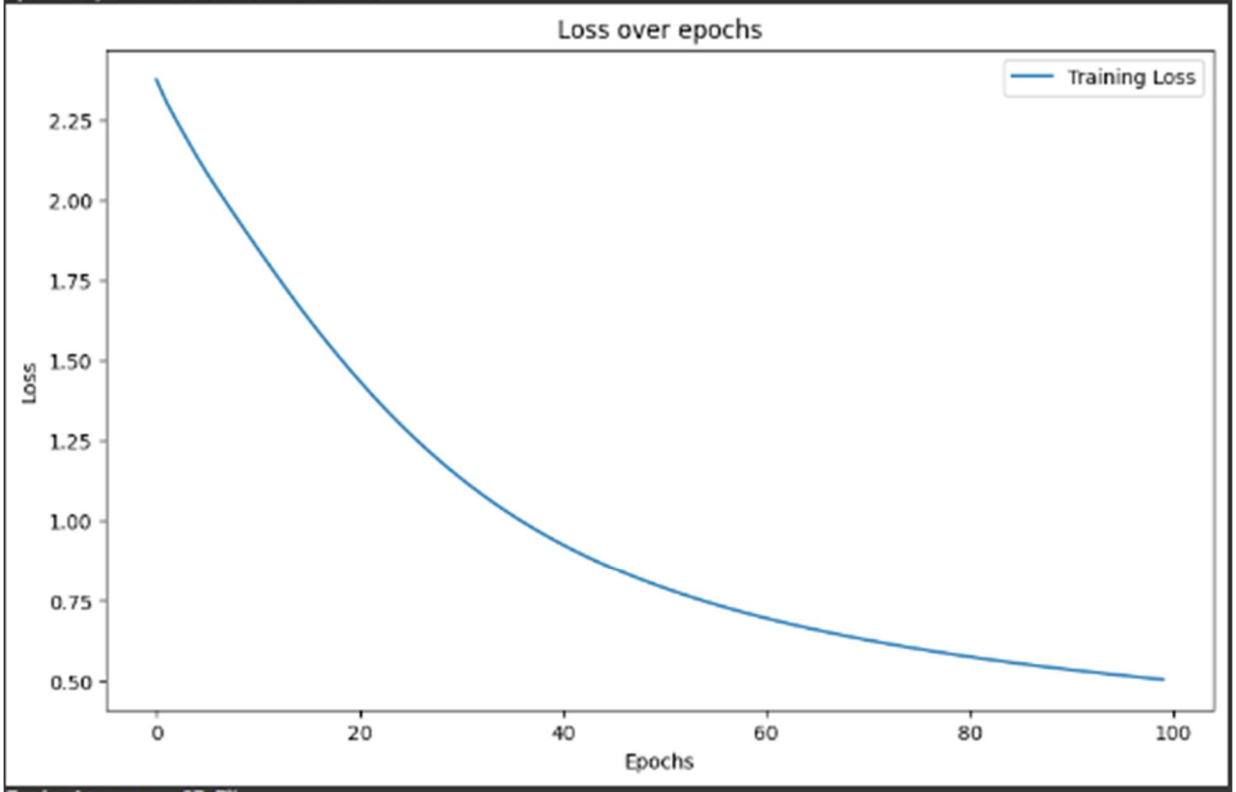
    def plot_loss(self):
        plt.figure(figsize=(10, 6))
        plt.plot(self.loss_history, label='Training Loss')
        plt.title('Loss over epochs')
        plt.xlabel('Epochs')
        plt.ylabel('Loss')
        plt.legend()
        plt.show()

# Creating and training the model
mlp = MultiLayerPerceptron(lr=0.1, iterations=100, n_hidden_neurons=25,
n_output_neurons=10, n_inputs=784)
mlp.train(x_train, y_train)

# Plotting the training loss
mlp.plot_loss()

# Evaluating the model
train_accuracy = mlp.accuracy(x_train, y_train)
test_accuracy = mlp.accuracy(x_test, y_test)
print(f'Train Accuracy: {train_accuracy}%')
print(f'Test Accuracy: {test_accuracy}%')
```

```
Epoch 0, Loss: 2.375757373879583
Epoch 10, Loss: 1.8476000751327697
Epoch 20, Loss: 1.4362533358441443
Epoch 30, Loss: 1.1316675293475862
Epoch 40, Loss: 0.9253954540600229
Epoch 50, Loss: 0.7882466700593764
Epoch 60, Loss: 0.694002212622089
Epoch 70, Loss: 0.6257966798326076
Epoch 80, Loss: 0.5740375178851604
Epoch 90, Loss: 0.5333616060834906
```




```
Train Accuracy: 87.7%
Test Accuracy: 84.6%
```